

Archive Digitale PFA

Technical Breakdown -- Full Project Analysis

1. Project Architecture

The project follows a clean two-layer architecture: a backend service layer (src/) and a GUI presentation layer (gui/), connected through direct Python imports (no HTTP, no event bus, no message queue).

Directory Structure

```
archive_pfa/
|-- main.py                <- DB bootstrap + seed accounts
|-- database/
|   +-- archive.db         <- SQLite file (single source of truth)
|-- storage/
|   +-- rapports/          <- Physical PDF files on disk
|-- src/                   <- Backend (pure logic, no UI)
|   |-- database.py        <- Connection factory + DDL (CREATE TABLE)
|   |-- crud.py            <- All DB reads/writes
|   |-- security.py        <- bcrypt hashing
|   +-- pdf_handler.py     <- PDF copy, text extraction, similarity
+-- gui/                   <- PyQt5 windows (pure UI)
    |-- login_window.py
    |-- main_window.py     <- Shell: sidebar nav + QStackedWidget
    |-- depot_window.py    <- Report submission form
    |-- evaluation_window.py <- Encadrant grading workflow
    |-- recherche_window.py <- Multi-criteria search
    |-- dashboard_window.py <- Statistics + pie chart
    |-- preview_window.py  <- Inline PDF viewer
    +-- users_window.py    <- Admin user CRUD
```

Note: main.py is NOT the application launcher -- it is a DB initializer/seeders. The actual GUI entry point is: `python -m gui.login_window`

2. Core Functionality

This is a desktop digital archive system for managing student end-of-year project reports (PFAs) at EMSI (Ecole Marocaine des Sciences de l'Ingenieur).

Feature	Description
Secure login	Email + bcrypt-hashed password, 3-attempt lockout
RBAC	3 roles with different menu visibility and data access
Report submission	PDF upload + metadata form + automatic plagiarism check
Plagiarism detection	TF-IDF + cosine similarity vs. all existing reports
Evaluation workflow	Encadrant reviews, grades, accepts or rejects reports
Search	Multi-criteria keyword/year search, role-filtered results

Archive Digitale PFA -- Technical Breakdown

PDF preview	Inline viewer with page navigation and zoom slider
Dashboard	Real-time stats cards + matplotlib pie chart
User management	Admin creates/edits/deletes all accounts

3. Technology Stack

Layer	Technology	Purpose
Language	Python 3.13	Entire application
GUI	PyQt5	All windows, forms, tables, dialogs
Database	SQLite 3	Embedded relational DB via sqlite3 stdlib
Security	bcrypt	Salted password hashing
PDF	PyMuPDF (fitz)	Text extraction + page rendering to pixmap
ML / NLP	scikit-learn	TfidfVectorizer + cosine_similarity
Charts	Matplotlib Qt5Agg	Pie chart embedded as Qt widget

4. Data Flow

4.1 Authentication Flow

```
User types email + password
|
v  LoginWindow.tenter_connexion()
|
+-- crud.get_utilisateur_by_email(email)
|     SELECT * FROM utilisateur WHERE email = ?
|
+-- security.verifier_mot_de_passe(input, stored_hash)
|     bcrypt.checkpw(bytes, hash_bytes) -> True / False
|
+-- on failure: increment tentatives counter (max 3, then disable button)
|
+-- on success: MainWindow(dict(user)) shown, LoginWindow hidden
                sidebar menu rendered based on user['role']
```

4.2 Report Submission Flow (most complex path)

```
Admin fills DepotWindow form -> clicks 'Deposer'
|
v  pdf_handler.deposer_rapport(chemin_source, titre, auteurs, ...)
|
+-- 1. copier_pdf()
|     Sanitize title -> 'ANNEE_TITRE.pdf'
|     Detect filename collision -> append counter suffix
|     shutil.copy2() to storage/rapports/
|
+-- 2. extraire_texte()
|     fitz.open(pdf) -> iterate pages -> page.get_text()
|     returns None if scanned image (no text layer)
```

```

|
+-- 3. analyser_similarite(texte_nouveau)
|     SELECT all rapports WHERE texte_extraite IS NOT NULL
|     TfidfVectorizer().fit_transform([new_text] + [all existing])
|     cosine_similarity(matrix[0:1], matrix[1:]) -> score array
|     returns list sorted by score desc, values in %
|
+-- 4. crud.ajouter_rapport()
|     INSERT INTO rapport (... , statut='Soumis')
|     returns rapport_id

```

DepotWindow.afficher_resultats_similarite(resultats)

Color codes: >= 70% = red | >= 40% = yellow | < 40% = green

4.3 Evaluation Flow

Encadrant selects report -> double-click or 'Evaluer'

```

|
v  EvaluationWindow.ouvrir_evaluation()
|
+-- If statut == 'Soumis': crud.changer_statut(id, 'En cours') <- auto-transition
|
+-- Opens EvaluationDialog (modal)
|
|   +-- Acceptor: evaluer_rapport(id, encadrant_id, note, comment, 'accepter')
|       Verifies rapport.encadrant_id == encadrant_id (ownership check)
|       UPDATE rapport SET statut='Archive', note=?, commentaire=?
|
|   +-- Rejeter: requires non-empty commentaire
|       UPDATE rapport SET statut='Rejete', note=?, commentaire=?

```

4.4 Search Flow (RBAC-filtered)

```

RechercheWindow.lancer_recherche()
|
+-- If role == 'etudiant':          statut_filtre = 'Archive'
+-- If role == 'encadrant' /
    'administrateur':              statut_filtre = None (see all)
|
v  crud.rechercher_rapports(mot_cle, promotion, option_id, statut_filtre)
    Dynamic SQL:  SELECT * FROM rapport WHERE 1=1
                  [AND titre / mots_cles / auteurs LIKE ?]
                  [AND promotion = ?]
                  [AND statut = ?]

```

5. Key Components

src/database.py -- Connection Factory + Schema

- get_connection(): opens SQLite with row_factory = sqlite3.Row, making all results accessible as row['column_name'] dicts instead of positional tuples.

- `create_tables()`: DDL using `CREATE TABLE IF NOT EXISTS`. The `utilisateur` table uses single-table inheritance: all three roles share one table with nullable role-specific columns (`numero_apogee`, `filiere`, `departement`, ...) enforced by a `CHECK` constraint on the `'role'` column.
- The `rapport` table's `statut` column has a `CHECK` constraint enforcing the state machine: `Soumis` -> `En cours` -> `Evalue` / `Rejete` / `Archive`.

src/crud.py -- Data Access Layer

- `ajouter_utilisateur()`: hashes the password before inserting via `hasher_mot_de_passe()`. Catches `sqlite3.IntegrityError` on duplicate email (`UNIQUE` constraint).
- `evaluer_rapport()`: performs an ownership check -- fetches `encadrant_id` from DB and compares it to the calling encadrant's ID, preventing cross-encadrant evaluation.
- `rechercher_rapports()`: builds dynamic SQL with `WHERE 1=1` + conditional `AND` clauses, safe from SQL injection via parameterized `?` placeholders.
- `get_statistiques()`: fires 6 separate `COUNT(*)` queries, one per status, for the dashboard.

src/security.py -- Password Security

- `hasher_mot_de_passe()`: calls `bcrypt.gensalt()` each time, producing a different hash for the same password on every call (salt is embedded in the output string).
- `verifier_mot_de_passe()`: uses `bcrypt.checkpw()` which extracts the salt from the stored hash internally -- no need to store the salt separately.

src/pdf_handler.py -- PDF + Plagiarism Engine

- `extraire_texte()`: uses PyMuPDF's `page.get_text()` across all pages. Returns `None` for scanned PDFs (image-only, no text layer).
- `analyser_similarite()`: builds a TF-IDF matrix where row 0 is the new document and rows 1..N are all existing stored texts. Computes cosine similarity of row 0 against all others. Result is a 0-1 float converted to a percentage. `Score >= 70%` = flagged red.
- `deposer_rapport()`: the orchestrator -- chains `copy` -> `extract` -> `similarity` -> DB insert, returning (`rapport_id`, `similarity_results`) to the GUI.

gui/main_window.py -- Application Shell

- Uses a `QStackedWidget` (`zone_contenu`) as the content area. Each nav click calls `changer_page()`, which destroys the old widget with `deleteLater()` and inserts a freshly constructed new one -- no page caching.
- The sidebar is built dynamically in `creer_sidebar()` with `'if role =='` guards, ensuring menu items only appear for authorized roles.
- The `utilisateur` dict (explicitly converted via `dict(utilisateur)` in `LoginWindow`) is passed through to all sub-windows as plain data.

gui/preview_window.py -- PDF Viewer

- Extends `QDialog` (modal). For each page renders via `page.get_pixmap(matrix=fitz.Matrix(zoom, zoom))` -- a zoom matrix scales the pixels.
- `Pixmap` bytes converted to `QImage` using `raw pix.samples` (`RGB888` format), wrapped in `QPixmap` and set on a `QLabel` inside a `QScrollArea`.
- `QSpinBox` signals are blocked during programmatic navigation (`blockSignals(True/False)`) to prevent recursive `valueChanged` triggers.
- `closeEvent()` explicitly calls `self.doc.close()` to release the file handle.

gui/evaluation_window.py -- Evaluation Workflow

- selection_changee(): re-fetches the full report row from DB on every table selection to ensure data freshness. Enables 'Evaluer' only if statut in ('Soumis', 'En cours').
- Auto-transition Soumis -> En cours happens silently when an encadrant opens the dialog, signaling to others that the report is being reviewed.
- EvaluationDialog enforces that a rejection always requires a non-empty comment before the CRUD call is made.

6. Report Status State Machine

```
[Submitted by admin]
  |
  v
Soumis
  | (encadrant opens evaluation dialog)
  v
En cours
  |
+----+----+
|      |      |
Archive  Rejete
(accepted) (rejected -- comment required)
```

Students (role = etudiant) can only read reports in the Archive state. Encadrants see only their own assigned reports in the evaluation screen. Administrators see everything across all states.

7. Notable Design Decisions

Decision	Consequence
No ORM -- raw sqlite3 throughout	Simple for this scale; all connection mgmt is manual (open/close per call).
Single-table inheritance for users	Avoids joins for the RBAC model, at the cost of many nullable columns.
Lazy GUI imports (inside methods)	Avoids circular imports and speeds up initial window load.
RBAC at two levels (UI sidebar + query filter + own)	Even if menu is bypassed, the DB layer still enforces access rules.
No session token	Authenticated user passed as a plain dict through the object graph.
QStackedWidget clears on every nav click	No stale state between views; each visit re-queries the DB fresh.